

13

Advanced GANs to Manipulate Images

In the previous chapter, we learned about leveraging **Generative Adversarial Networks (GANs)** to generate realistic images. In this chapter, we will learn about leveraging GANs to manipulate images. We will learn about two variations of generating images using GANs – paired and unpaired methods. With the paired method, we will provide the input and output pair combinations to generate novel images based on an input image, which we will learn about in the **Pix2Pix GAN**. With the unpaired method, we will specify the input and output; however, we will not provide one-to-one correspondence between the input and output, but expect the GAN to learn the structure of the two classes, and convert an image from one class to another, which we will learn about when we discuss **CycleGAN**.

Another class of unpaired image manipulation involves generating images from a latent space of random vectors and seeing how images change as the latent vector values change, which we will learn about in the *Leveraging StyleGAN on custom images* section. Finally, we will learn about leveraging a pre-trained GAN – **Super Resolution Generative Adversarial Network (SRGAN)**, with which we can turn a low-resolution image into an image with high resolution.

Specifically, we will learn about the following topics:

- Leveraging the Pix2Pix GAN to convert a sketch/ picture of edges to a picture
- Leveraging CycleGAN to convert apples to oranges and vice versa
- Leveraging StyleGAN on custom images to change the expression of images
- Super-resolution of an image using SRGAN



All code snippets within this chapter are available in the **Chapter13** folder of the GitHub repository at <https://bit.ly/mcvcv-2e>.

Leveraging the Pix2Pix GAN

Imagine a scenario where we have pairs of images that are related to each other (for example, an image of the edges of an object as input and an actual image of the object as output). The challenge given is that we want to generate an image given the input image of the edges of an object. In a traditional setting, this would have been a simple mapping of input to output and hence a supervised learning problem. However, imagine that you are working with a creative team that is trying to come up with a fresh look for products. In this scenario, supervised learning does not help much as it only learns from history. A GAN would come in handy

here because it would ensure that the generated image would look realistic and would leave room for experimentation (as we are interested in checking whether the generated image is similar to the images that we want to generate). Specifically, Pix2Pix GAN comes in handy in scenarios in which it is trained to generate an image from another image that only contains its edges (or contours).

In this section, we will learn about the architecture used to generate an image of a shoe from a hand-drawn doodle of the edges of a shoe. The strategy that we will adopt to generate a realistic image from the doodle is as follows:

1. Fetch a lot of actual images and create the corresponding edges using standard `cv2` edge detection techniques.
2. Sample colors from patches of the original image so that the generator knows which colors to generate.
3. Build a UNet architecture that takes the edges with the sample patch colors as input and predicts the corresponding image – this is our generator.
4. Build a discriminator architecture that takes an image and predicts whether it is real or fake.
5. Train the generator and discriminator together to a point where the generator can generate images that fool the discriminator.

Let's code the strategy:



The following code is available as `Pix2Pix_GAN.ipynb` in the `Chapter13` folder of this book's GitHub repository: <https://bit.ly/mcvc-2e>. The code contains URLs to download data from and is moderately lengthy. We strongly recommend you execute the notebook in GitHub to reproduce the results while you follow the steps to perform and read the explanations of the various code components in the text.

1. Import the dataset (available here: <https://sketchx.eecs.qmul.ac.uk/downloads/>) and install the relevant packages:

```
!wget https://www.dropbox.com/s/g6b6gtvmd0h77x/ShoeV2_photo.zip
!pip install torch_snippets
!pip install torch_summary
from torch_snippets import *
device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

The preceding code downloads images of shoes. A sample of the downloaded images is as follows:



Figure 13.1: Sample images

For our problem, we want to draw a shoe given the edges and some sample patch colors of the shoe. In the next step, we will fetch the edges from an image of a shoe. This way, we can train a model that reconstructs an image of a shoe from the edges and sample patch colors of the shoe.

2. Define a function to fetch the edges from the downloaded images:

```
def detect_edges(img):
    img_gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
    img_gray = cv2.bilateralFilter(img_gray, 5, 50, 50)
    img_gray_edges = cv2.Canny(img_gray, 45, 100)
    # invert black/white
    img_gray_edges = cv2.bitwise_not(img_gray_edges)
    img_edges = cv2.cvtColor(img_gray_edges, cv2.COLOR_GRAY2RGB)
    return img_edges
```

In the preceding code, we are leveraging the various methods available in the OpenCV package to fetch the edges from an image (there are more details on how the OpenCV methods work in the OpenCV chapter within the `Extra chapters from first edition` folder in the GitHub repository).

3. Define the image transformation pipeline to pre-process and normalize the dataset (`preprocess` and `normalize`):

```
IMAGE_SIZE = 256
preprocess = T.Compose([
    T.Lambda(lambda x: torch.Tensor(x.copy())\
        .permute(2, 0, 1).to(device))])
normalize = lambda x: (x - 127.5)/127.5
```

4. Define the dataset class (`ShoesData`). This dataset class returns the original image and the image with edges. One additional detail we will pass to the network is some patches of color that are present in randomly chosen regions. This way, we are enabling the user to take a hand-drawn contour image, sprinkle the required colors in different parts of the image, and generate a new image.

An example input (the first image) and output (the third image) are shown here (you can view them in color in the digital version of this book):



Figure 13.2: (Left) Original image; (middle) contours of the original image; (right) contour image with color information

However, the input image we have is just of the shoe (the leftmost image), which we will use to extract the edges of the shoe (the middle image). Further, we will sprinkle color in the next step to fetch the color

information of the original image (the rightmost image). The output of the rightmost image, when passed through our network, should be the leftmost image. In the following code, we will build the class that takes the contour images, sprinkles colors, and returns the pair of color-sprinkled images and the original shoe image (the image that generated the image with contours):

1. Define the `ShoesData` class, the `__init__` method, and the `__len__` method:

```
class ShoesData(Dataset):
    def __init__(self, items):
        self.items = items
    def __len__(self): return len(self.items)
```

2. Define the `__getitem__` method. In this method, we will process the input image to fetch an image with edges and then sprinkle the image with the colors present in the original image. Here, we are fetching the edges of a given image:

```
def __getitem__(self, ix):
    f = self.items[ix]
    try: im = read(f, 1)
    except:
        blank = preprocess(Blank(IMAGE_SIZE, IMAGE_SIZE, 3))
        return blank, blank
    edges = detect_edges(im)
```

3. Once we have fetched the edges in the image, we resize and normalize the image:

```
im, edges = resize(im, IMAGE_SIZE), resize(edges, IMAGE_SIZE)
im, edges = normalize(im), normalize(edges)
```

4. Sprinkle color on the `edges` image and preprocess the original and `edges` images:

```
self._draw_color_circles_on_src_img(edges, im)
im, edges = preprocess(im), preprocess(edges)
return edges, im
```

5. Define the functions to sprinkle color:

```
def _draw_color_circles_on_src_img(self, img_src,
                                   img_target):
    non_white_coords = self._get_non_white_coordinates(img_target)
    for center_y, center_x in non_white_coords:
        self._draw_color_circle_on_src_img(img_src,
                                             img_target, center_y, center_x)
    def _get_non_white_coordinates(self, img):
        non_white_mask = np.sum(img, axis=-1) < 2.75
        non_white_y, non_white_x = np.nonzero(non_white_mask)
        # randomly sample non-white coordinates
```

```

n_non_white = len(non_white_y)
n_color_points = min(n_non_white, 300)
idxs = np.random.choice(n_non_white, n_color_points,
                        replace=False)

non_white_coords = list(zip(non_white_y[idxs],
                           non_white_x[idxs]))

return non_white_coords

def _draw_color_circle_on_src_img(self, img_src,
                                img_target, center_y, center_x):
    assert img_src.shape == img_target.shape
    y0, y1, x0, x1 = self._get_color_point_bbox_coords(center_y,
                                                         color= np.mean(img_target[y0:y1, x0:x1], axis=(0, 1))
    img_src[y0:y1, x0:x1] = color
def _get_color_point_bbox_coords(self, center_y, center_x):
    radius = 2
    y0 = max(0, center_y-radius+1)
    y1 = min(IMAGE_SIZE, center_y+radius)
    x0 = max(0, center_x-radius+1)
    x1 = min(IMAGE_SIZE, center_x+radius)
    return y0, y1, x0, x1
def choose(self): return self[randint(len(self))]

```

5. Define the training and validation data's corresponding datasets and data loaders:

```

from sklearn.model_selection import train_test_split
train_items, val_items = train_test_split(Glob('ShoeV2_photo/*.png'),
                                         test_size=0.2, random_state=2)
trn_ds, val_ds = ShoesData(train_items), ShoesData(val_items)
trn_dl = DataLoader(trn_ds, batch_size=32, shuffle=True)
val_dl = DataLoader(val_ds, batch_size=32, shuffle=True)

```

6. Define the generator and discriminator architectures, which use the weight initialization (`weights_init_normal`), `UNetDown`, and `UNetUp` architectures, just as we did in *Chapter 9, Image Segmentation*, and *Chapter 10, Applications of Object Detection and Segmentation*, to define the `GeneratorUNet` and `Discriminator` architectures:

1. Initialize weights so that they follow a normal distribution:

```

def weights_init_normal(m):
    classname = m.__class__.__name__
    if classname.find("Conv") != -1:
        torch.nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find("BatchNorm2d") != -1:
        torch.nn.init.normal_(m.weight.data, 1.0, 0.02)
        torch.nn.init.constant_(m.bias.data, 0.0)

```

2. Define the `UnetDown` and `UNetUp` classes:

```

class UNetDown(nn.Module):
    def __init__(self, in_size, out_size, normalize=True, dropout=0.0):
        super(UNetDown, self).__init__()
        layers = [nn.Conv2d(in_size, out_size, 4, 2, 1, bias=False)]
        if normalize:
            layers.append(nn.InstanceNorm2d(out_size))
        layers.append(nn.LeakyReLU(0.2))

```

```

        if dropout:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x):
        return self.model(x)
class UNetUp(nn.Module):
    def __init__(self, in_size, out_size, dropout=0.0):
        super(UNetUp, self).__init__()
        layers = [
            nn.ConvTranspose2d(in_size, out_size, 4, 2, 1, bias=False),
            nn.InstanceNorm2d(out_size),
            nn.ReLU(inplace=True),
        ]
        if dropout:
            layers.append(nn.Dropout(dropout))
        self.model = nn.Sequential(*layers)
    def forward(self, x, skip_input):
        x = self.model(x)
        x = torch.cat((x, skip_input), 1)
        return x

```

3. Define the `GeneratorUNet` class:

```

class GeneratorUNet(nn.Module):
    def __init__(self, in_channels=3, out_channels=3):
        super(GeneratorUNet, self).__init__()
        self.down1 = UNetDown(in_channels, 64, normalize=False)
        self.down2 = UNetDown(64, 128)
        self.down3 = UNetDown(128, 256)
        self.down4 = UNetDown(256, 512, dropout=0.5)
        self.down5 = UNetDown(512, 512, dropout=0.5)
        self.down6 = UNetDown(512, 512, dropout=0.5)
        self.down7 = UNetDown(512, 512, dropout=0.5)
        self.down8 = UNetDown(512, 512, normalize=False, dropout=0.5)
        self.up1 = UNetUp(512, 512, dropout=0.5)
        self.up2 = UNetUp(1024, 512, dropout=0.5)
        self.up3 = UNetUp(1024, 512, dropout=0.5)
        self.up4 = UNetUp(1024, 512, dropout=0.5)
        self.up5 = UNetUp(1024, 256)
        self.up6 = UNetUp(512, 128)
        self.up7 = UNetUp(256, 64)
        self.final = nn.Sequential(
            nn.Upsample(scale_factor=2),
            nn.ZeroPad2d((1, 0, 1, 0)),
            nn.Conv2d(128, out_channels, 4, padding=1),
            nn.Tanh(),
        )
    def forward(self, x):
        d1 = self.down1(x)
        d2 = self.down2(d1)
        d3 = self.down3(d2)
        d4 = self.down4(d3)
        d5 = self.down5(d4)
        d6 = self.down6(d5)
        d7 = self.down7(d6)
        d8 = self.down8(d7)
        u1 = self.up1(d8, d7)
        u2 = self.up2(u1, d6)
        u3 = self.up3(u2, d5)
        u4 = self.up4(u3, d4)

```

```

u5 = self.up5(u4, d3)
u6 = self.up6(u5, d2)
u7 = self.up7(u6, d1)
return self.final(u7)

```

4. Define the `Discriminator` class:

```

class Discriminator(nn.Module):
    def __init__(self, in_channels=3):
        super(Discriminator, self).__init__()
        def discriminator_block(in_filters, out_filters,
                                normalization=True):
            """Returns downsampling layers of each
            discriminator block"""
            layers = [nn.Conv2d(in_filters, out_filters, 4,
                                stride=2, padding=1)]

            if normalization:
                layers.append(nn.InstanceNorm2d(out_filters))
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *discriminator_block(in_channels * 2, 64,
                                normalization=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.ZeroPad2d((1, 0, 1, 0)),
            nn.Conv2d(512, 1, 4, padding=1, bias=False)
        )

    def forward(self, img_A, img_B):
        img_input = torch.cat((img_A, img_B), 1)
        return self.model(img_input)

```

7. Define the `generator` and `discriminator` model objects and fetch summaries:

```

generator = GeneratorUNet().to(device)
discriminator = Discriminator().to(device)
from torchsummary import summary
print(summary(generator, torch.zeros(3, 3, IMAGE_SIZE,
                                     IMAGE_SIZE).to(device)))
print(summary(discriminator, torch.zeros(3, 3, IMAGE_SIZE, IMAGE_SIZE).to(device), torch.zeros(

```

The generator architecture summary is as follows:

Layer (type)	Output Shape	Param #	Tr. Param #
UNetDown-1	[3, 64, 128, 128]	3,072	3,072
UNetDown-2	[3, 128, 64, 64]	131,072	131,072
UNetDown-3	[3, 256, 32, 32]	524,288	524,288
UNetDown-4	[3, 512, 16, 16]	2,097,152	2,097,152
UNetDown-5	[3, 512, 8, 8]	4,194,304	4,194,304
UNetDown-6	[3, 512, 4, 4]	4,194,304	4,194,304
UNetDown-7	[3, 512, 2, 2]	4,194,304	4,194,304
UNetDown-8	[3, 512, 1, 1]	4,194,304	4,194,304
UNetUp-9	[3, 1024, 2, 2]	4,194,304	4,194,304
UNetUp-10	[3, 1024, 4, 4]	8,388,608	8,388,608
UNetUp-11	[3, 1024, 8, 8]	8,388,608	8,388,608
UNetUp-12	[3, 1024, 16, 16]	8,388,608	8,388,608
UNetUp-13	[3, 512, 32, 32]	4,194,304	4,194,304
UNetUp-14	[3, 256, 64, 64]	1,048,576	1,048,576
UNetUp-15	[3, 128, 128, 128]	262,144	262,144
Upsample-16	[3, 128, 256, 256]	0	0
ZeroPad2d-17	[3, 128, 257, 257]	0	0
Conv2d-18	[3, 3, 256, 256]	6,147	6,147
Tanh-19	[3, 3, 256, 256]	0	0
Total params: 54,404,099			
Trainable params: 54,404,099			
Non-trainable params: 0			

Figure 13.3: Summary of the generator architecture

The discriminator architecture summary is as follows:

Layer (type)	Output Shape	Param #	Tr. Param #
Conv2d-1	[3, 64, 128, 128]	6,208	6,208
LeakyReLU-2	[3, 64, 128, 128]	0	0
Conv2d-3	[3, 128, 64, 64]	131,200	131,200
InstanceNorm2d-4	[3, 128, 64, 64]	0	0
LeakyReLU-5	[3, 128, 64, 64]	0	0
Conv2d-6	[3, 256, 32, 32]	524,544	524,544
InstanceNorm2d-7	[3, 256, 32, 32]	0	0
LeakyReLU-8	[3, 256, 32, 32]	0	0
Conv2d-9	[3, 512, 16, 16]	2,097,664	2,097,664
InstanceNorm2d-10	[3, 512, 16, 16]	0	0
LeakyReLU-11	[3, 512, 16, 16]	0	0
ZeroPad2d-12	[3, 512, 17, 17]	0	0
Conv2d-13	[3, 1, 16, 16]	8,192	8,192
Total params: 2,767,808			
Trainable params: 2,767,808			
Non-trainable params: 0			

Figure 13.4: Summary of the discriminator architecture

8. Define the function to train the discriminator

(discriminator_train_step):

1. The discriminator function takes the source image (`real_src`), real target (`real_trg`), and fake target (`fake_trg`) as input:

```
def discriminator_train_step(real_src, real_trg, fake_trg):
    d_optimizer.zero_grad()
```

2. Calculate the loss (`error_real`) by comparing the real target (`real_trg`) and the predicted values (`real_src`) of the target; the expectation is that the discriminator will predict the images as real (indicated by `torch.ones`) and then perform backpropagation:

```
prediction_real = discriminator(real_trg, real_src)
error_real = criterion_GAN(prediction_real,
                           torch.ones(len(real_src),
```



```

1, 16, 16).to(device))
error_real.backward()

```

3. Calculate the discriminator loss (`error_fake`) corresponding to the fake images (`fake_trg`); the expectation is that the discriminator classifies the fake targets as fake images (indicated by `torch.zeros`) and then performs backpropagation:

```

prediction_fake = discriminator( real_src, fake_trg.detach())
error_fake = criterion_GAN(prediction_fake,
                           torch.zeros(len(real_src), 1,
                                       16, 16).to(device))
error_fake.backward()

```

4. Perform the optimizer step and return the overall error and loss values on the predicted real and fake targets:

```

d_optimizer.step()
return error_real + error_fake

```

9. Define the function to train the generator (`generator_train_step`), which takes a fake target (`fake_trg`) and trains it so that it has a low chance of being identified as fake when passed through the discriminator:

```

def generator_train_step(real_src, fake_trg):
    g_optimizer.zero_grad()
    prediction = discriminator(fake_trg, real_src)
    loss_GAN = criterion_GAN(prediction,
                             torch.ones(len(real_src), 1, 16, 16).to(device))
    loss_pixel = criterion_pixelwise(fake_trg, real_trg)
    loss_G = loss_GAN + lambda_pixel * loss_pixel
    loss_G.backward()
    g_optimizer.step()
    return loss_G

```

Note that in the preceding code, in addition to generator loss, we are also fetching the pixel loss (`loss_pixel`) corresponding to the difference between the generated and the real image of a given contour.

10. Define a function to fetch a sample of predictions:

```

denorm = T.Normalize((-1, -1, -1), (2, 2, 2))
def sample_prediction():
    """Saves a generated sample from the validation set"""
    data = next(iter(val_dl))
    real_src, real_trg = data
    fake_trg = generator(real_src)
    img_sample = torch.cat([denorm(real_src[0]),
                           denorm(fake_trg[0]),
                           denorm(real_trg[0])], -1)
    img_sample = img_sample.detach().cpu().permute(1,2,0).numpy()
    show(img_sample, title='Source::Generated::GroundTruth',sz=12)

```

11. Apply weight initialization (`weights_init_normal`) to the generator and discriminator model objects:

```
generator.apply(weights_init_normal)
discriminator.apply(weights_init_normal)
```

12. Specify the loss criterion and optimization methods (`criterion_GAN` and `criterion_pixelwise`):

```
criterion_GAN = torch.nn.MSELoss()
criterion_pixelwise = torch.nn.L1Loss()
lambda_pixel = 100
g_optimizer = torch.optim.Adam(generator.parameters(),
                                lr=0.0002, betas=(0.5, 0.999))
d_optimizer = torch.optim.Adam(discriminator.parameters(),
                                lr=0.0002, betas=(0.5, 0.999))
```

13. Train the model over 100 epochs:

```
epochs = 100
log = Report(epochs)
for epoch in range(epochs):
    N = len(trn_dl)
    for bx, batch in enumerate(trn_dl):
        real_src, real_trg = batch
        fake_trg = generator(real_src)
        errD = discriminator_train_step(real_src, real_trg, fake_trg)
        errG = generator_train_step(real_src, fake_trg)
        log.record(pos=epoch+(1+bx)/N, errD=errD.item(),
                  errG=errG.item(), end='\r')
    [sample_prediction() for _ in range(2)]
```

14. Generate the output (image) on a sample hand-drawn contour:

```
[sample_prediction() for _ in range(2)]
```

The preceding code generates the following output:



Figure 13.5: (Left) Input image; (middle) generated image; (right) original image

Note that in the preceding output, we have generated images that have similar colors as those of the original image. As an exercise, we encourage you to train the model for more epochs and see the improvement in generated images.

In this section, we have learned about using the contours of an image to generate an image. However, this required us to provide the input and output as pairs, which can be a tedious process. In the next section, we will learn about unpaired image translation. This is a process by which the network figures out the translation without needing us to specify the input and output mappings of images.

Leveraging CycleGAN

Imagine a scenario where we ask you to perform image translation from one class to another, but without using the input and the corresponding output images to train the model. For example, change the actor present in the current scene of a movie from one actor another. However, we give you the images of both classes/actors in two distinct folders. CycleGAN comes in handy in such a scenario.

In this section, we will learn how to train CycleGAN

(<https://arxiv.org/abs/1703.10593>) to convert an image of an apple into an image of an orange and vice versa. But first, let's understand how CycleGAN works.

How CycleGAN works

The **Cycle** in CycleGAN refers to the fact that we are translating (converting) an image from one class to another and back to the original class. At a high level, we will have three separate loss values in this architecture. More details about the loss calculations in the next pages:

- **Adversarial loss:** This ensures that the both the domain generators accurately create objects in their respective domains using the other domain images as inputs. The only difference from a standard GAN, in this case, is that the generators accept images instead of noise.
- **Cycle loss:** The loss of recycling an image from the generated image to the original to ensure that the surrounding pixels are not changed.
- **Identity loss:** The loss when an input image of one class is passed through a generator that is expected to convert an image of another class into the class of the input image.

Here, we will go through the high-level steps of building CycleGAN:

1. Import and preprocess the dataset.
2. Build the generator and discriminator network UNet architectures.
3. Define two generators:
 1. **G_AB:** A generator that converts an image of class A to an image of class B
 2. **G_BA:** A generator that converts an image of class B to an image of class A
4. Define the **identity loss**:
 1. If you were to send an orange image to an orange generator, ideally if the generator has understood everything about oranges, it should not change the image and should "generate" the exact same image. We thus create an identity using this knowledge.
 2. Identity loss should be minimal when an image of class A (real_A) is passed through G_BA and compared with real_A.
 3. Identity loss should be minimal when an image of class B (real_B) is passed through G_AB and compared with real_B.
5. Define the **GAN loss**:
 1. The discriminator and generator loss for real_A and fake_A (fake_A is obtained when real_B image is passed through G_BA)

2. The discriminator and generator loss for real_B and fake_B (fake_B is obtained when the real_A image is passed through G_AB)
6. Define the **re-cycle loss**:
 1. Consider a scenario where an image of an apple is to be transformed by an orange generator to generate a fake orange, and the fake orange is to be transformed back into an apple by the apple generator.
 2. fake_B, which is the output when real_A is passed through G_AB, should regenerate real_A when fake_B is passed through G_BA.
 3. fake_A, which is the output when real_B is passed through G_BA, should regenerate real_B when fake_A is passed through G_AB.
7. Optimize for the weighted loss of the three losses.

Now that we understand the steps, let's code them to convert apples to oranges and vice versa.

Implementing CycleGAN

To implement the steps that we just discussed, you can use the following code:



This code is available as `CycleGAN.ipynb` in the `Chapter13` folder of this book's GitHub repository:
<https://bit.ly/mcvc-2e>. The code contains URLs to download data from and is moderately lengthy. We strongly recommend you execute the notebook in GitHub to reproduce results while you read the steps to perform and the explanations of the code components in the text.

1. Download and extract the datasets that contain the folders that have the images of apples and oranges:

```
!wget https://www.dropbox.com/s/2xltmolfbfharri/apples_oranges.zip
!unzip apples_oranges.zip
```

Here's a sample of the images we will be working on:

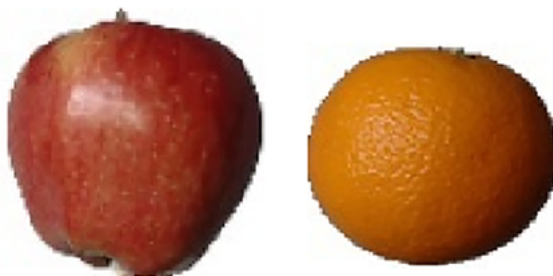


Figure 13.6: Sample images

Note that there is no one-to-one correspondence between the apple and orange images (unlike the contour-to-shoe generation use case that we learned about in the *Leveraging the Pix2Pix GAN* section).

2. Import the required packages:

```
!pip install torch_snippets torch_summary
import itertools
from PIL import Image
from torch_snippets import *
from torchvision import transforms
from torchvision.utils import make_grid
from torchsummary import summary
```

3. Define the image transformation pipeline (`transform`):

```
IMAGE_SIZE = 256
device = 'cuda' if torch.cuda.is_available() else 'cpu'
transform = transforms.Compose([
    transforms.Resize(int(IMAGE_SIZE*1.33)),
    transforms.RandomCrop((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
```

4. Define the dataset class (`CycleGANDataset`), which takes the `apple` and `orange` folders (which are obtained after unzipping the downloaded dataset) as input and provides a batch of apple and orange images:

```
class CycleGANDataset(Dataset):
    def __init__(self, apples, oranges):
        self.apples = Glob(apples)
        self.oranges = Glob(oranges)
    def __getitem__(self, ix):
        apple = self.apples[ix % len(self.apples)]
        orange = choose(self.oranges)
        apple = Image.open(apple).convert('RGB')
        orange = Image.open(orange).convert('RGB')
        return apple, orange
    def __len__(self): return max(len(self.apples), len(self.oranges))
    def choose(self): return self[randint(len(self))]
    def collate_fn(self, batch):
        srcs, trgs = list(zip(*batch))
        srcs=torch.cat([transform(img)[None] for img in \
                        srcs], 0).to(device).float()
        trgs=torch.cat([transform(img)[None] for img in \
                        trgs], 0).to(device).float()
        return srcs.to(device), trgs.to(device)
```

5. Define the training and validation datasets and data loaders:

```
trn_ds = CycleGANDataset('apples_train', 'oranges_train')
val_ds = CycleGANDataset('apples_test', 'oranges_test')
trn_dl = DataLoader(trn_ds, batch_size=1, shuffle=True,
                   collate_fn=trn_ds.collate_fn)
val_dl = DataLoader(val_ds, batch_size=5, shuffle=True,
                   collate_fn=val_ds.collate_fn)
```

6. Define the weight initialization method of the network
(`weights_init_normal`) as defined in previous sections:

```
def weights_init_normal(m):
    classname = m.__class__.__name__
    if classname.find("Conv") != -1:
        torch.nn.init.normal_(m.weight.data, 0.0, 0.02)
        if hasattr(m, "bias") and m.bias is not None:
            torch.nn.init.constant_(m.bias.data, 0.0)
    elif classname.find("BatchNorm2d") != -1:
        torch.nn.init.normal_(m.weight.data, 1.0, 0.02)
        torch.nn.init.constant_(m.bias.data, 0.0)
```

7. Define the residual block network (`ResidualBlock`) because in this instance, we will use ResNet:

```
class ResidualBlock(nn.Module):
    def __init__(self, in_features):
        super(ResidualBlock, self).__init__()
        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features),
            nn.ReLU(inplace=True),
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features),
        )
    def forward(self, x):
        return x + self.block(x)
```

8. Define the generator network (`GeneratorResNet`):

```
class GeneratorResNet(nn.Module):
    def __init__(self, num_residual_blocks=9):
        super(GeneratorResNet, self).__init__()
        out_features = 64
        channels = 3
        model = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(channels, out_features, 7),
            nn.InstanceNorm2d(out_features),
            nn.ReLU(inplace=True),
        ]
        in_features = out_features
        # Downsampling
        for _ in range(2):
            out_features *= 2
            model += [
                nn.Conv2d(in_features, out_features, 3,
                           stride=2, padding=1),
                nn.InstanceNorm2d(out_features),
                nn.ReLU(inplace=True),
            ]
            in_features = out_features
        # Residual blocks
        for _ in range(num_residual_blocks):
```

```

        model += [ResidualBlock(out_features)]
    # Upsampling
    for _ in range(2):
        out_features //= 2
        model += [
            nn.Upsample(scale_factor=2),
            nn.Conv2d(in_features, out_features, 3,
                      stride=1, padding=1),
            nn.InstanceNorm2d(out_features),
            nn.ReLU(inplace=True),
        ]
        in_features = out_features
    # Output Layer
    model += [nn.ReflectionPad2d(channels),
              nn.Conv2d(out_features, channels, 7),
              nn.Tanh()]
    self.model = nn.Sequential(*model)
    self.apply(weights_init_normal)
    def forward(self, x):
        return self.model(x)

```

9. Define the discriminator network (`Discriminator`):

```

class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        channels, height, width = 3, IMAGE_SIZE, IMAGE_SIZE
        def discriminator_block(in_filters, out_filters,
                                normalize=True):
            """Returns downsampling layers of each
            discriminator block"""
            layers = [nn.Conv2d(in_filters, out_filters, 4,
                                stride=2, padding=1)]
            if normalize:
                layers.append(nn.InstanceNorm2d(out_filters))
                layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers
        self.model = nn.Sequential(
            *discriminator_block(channels, 64, normalize=False),
            *discriminator_block(64, 128),
            *discriminator_block(128, 256),
            *discriminator_block(256, 512),
            nn.ZeroPad2d((1, 0, 1, 0)),
            nn.Conv2d(512, 1, 4, padding=1)
        )
        self.apply(weights_init_normal)
    def forward(self, img):
        return self.model(img)

```

10. Define the function to generate a sample of images –

`generate_sample`:

```

@torch.no_grad()
def generate_sample():
    data = next(iter(val_dl))
    G_AB.eval()
    G_BA.eval()
    real_A, real_B = data
    fake_B = G_AB(real_A)

```



```

fake_A = G_BA(real_B)
# Arrange images along x-axis
real_A = make_grid(real_A, nrow=5, normalize=True)
real_B = make_grid(real_B, nrow=5, normalize=True)
fake_A = make_grid(fake_A, nrow=5, normalize=True)
fake_B = make_grid(fake_B, nrow=5, normalize=True)
# Arrange images along y-axis
image_grid = torch.cat((real_A, fake_B, real_B, fake_A), 1)
show(image_grid.detach().cpu().permute(1,2,0).numpy(),sz=12)

```

11. Define the function to train the generator (`generator_train_step`):

1. The function takes the two generator models (`G_AB` and `G_BA`) as `Gs` and `optimizer` , and real images of the two classes, `real_A` and `real_B` , as input:

```
def generator_train_step(Gs, optimizer, real_A, real_B):
```

2. Specify the generators:

```
G_AB, G_BA = Gs
```

3. Set the gradients to zero for the optimizer:

```
optimizer.zero_grad()
```

4. If you were to send an orange image to an orange generator, ideally, if the generator has understood everything about oranges, it should not make any changes to the image and should “generate” the exact image. We thus create an identity using this knowledge. The loss function corresponding to `criterion_identity` will be defined just prior to training the model. Calculate the identity loss (`loss_identity`) for images of type A (apples) and type B (oranges):

```

loss_id_A = criterion_identity(G_BA(real_A), real_A)
loss_id_B = criterion_identity(G_AB(real_B), real_B)
loss_identity = (loss_id_A + loss_id_B) / 2

```

5. Calculate the GAN loss when the image is passed through the generator and the generated image is expected to be as close to the other class as possible (we have `np.ones` in this case when training the generator, as we are passing the fake images of a class to the discriminator of the same class):

```

fake_B = G_AB(real_A)
loss_GAN_AB = criterion_GAN(D_B(fake_B),
                             torch.Tensor(np.ones((len(real_A), 1,
                                                     16, 16))).to(device))

fake_A = G_BA(real_B)
loss_GAN_BA = criterion_GAN(D_A(fake_A),
                             torch.Tensor(np.ones((len(real_A), 1,
                                                     16, 16))).to(device))

loss_GAN = (loss_GAN_AB + loss_GAN_BA) / 2

```

6. Calculate the cycle loss. Consider a scenario in which an image of an apple is to be transformed by an orange generator to generate a fake orange, and this fake orange is to be transformed back into an apple by the apple generator. If the generators were perfect, this process should give back the original image, which means the following cycle losses should be zero:

```
recov_A = G_BA(fake_B)
loss_cycle_A = criterion_cycle(recov_A, real_A)
recov_B = G_AB(fake_A)
loss_cycle_B = criterion_cycle(recov_B, real_B)
loss_cycle = (loss_cycle_A + loss_cycle_B) / 2
```

7. Calculate the overall weighted loss (where `lambda_cyc` and `lambda_id` are the weights associated with cycle loss and identity loss respectively) and perform backpropagation before returning the calculated values:

```
loss_G = loss_GAN + lambda_cyc * loss_cycle + \
        lambda_id * loss_identity
loss_G.backward()
optimizer.step()
return loss_G, loss_identity, loss_GAN, loss_cycle,
        loss_G, fake_A, fake_B
```

12. Define the function to train the discriminator
(`discriminator_train_step`):

```
def discriminator_train_step(D, real_data, fake_data, optimizer):
    optimizer.zero_grad()
    loss_real = criterion_GAN(D(real_data),
                              torch.Tensor(np.ones((len(real_data), 1,
                                                    16, 16))).to(device))
    loss_fake = criterion_GAN(D(fake_data.detach()),
                              torch.Tensor(np.zeros((len(real_data), 1,
                                                    16, 16))).to(device))
    loss_D = (loss_real + loss_fake) / 2
    loss_D.backward()
    optimizer.step()
    return loss_D
```

13. Define the generator, discriminator objects, optimizers, and loss functions:

```
G_AB = GeneratorResNet().to(device)
G_BA = GeneratorResNet().to(device)
D_A = Discriminator().to(device)
D_B = Discriminator().to(device)
criterion_GAN = torch.nn.MSELoss()
criterion_cycle = torch.nn.L1Loss()
criterion_identity = torch.nn.L1Loss()
optimizer_G = torch.optim.Adam(
    itertools.chain(G_AB.parameters(), G_BA.parameters()),
    lr=0.0002, betas=(0.5, 0.999))
optimizer_D_A = torch.optim.Adam(D_A.parameters(),
                                  lr=0.0002, betas=(0.5, 0.999))
```

```
optimizer_D_B = torch.optim.Adam(D_B.parameters(),
                                  lr=0.0002, betas=(0.5, 0.999))
lambda_cyc, lambda_id = 10.0, 5.0
```

14. Train the networks over increasing epochs:

```
n_epochs = 10
log = Report(n_epochs)
for epoch in range(n_epochs):
    N = len(trn_dl)
    for bx, batch in enumerate(trn_dl):
        real_A, real_B = batch
        loss_G, loss_identity, loss_GAN, loss_cycle, \
        loss_G, fake_A, fake_B = generator_train_step(\
            (G_AB, G_BA), optimizer_G,
            real_A, real_B)
        loss_D_A = discriminator_train_step(D_A, real_A,
            fake_A, optimizer_D_A)
        loss_D_B = discriminator_train_step(D_B, real_B,
            fake_B, optimizer_D_B)
        loss_D = (loss_D_A + loss_D_B) / 2

        log.record(epoch+(1+bx)/N, loss_D=loss_D.item(),
                    loss_G=loss_G.item(), loss_GAN=loss_GAN.item(),
                    loss_cycle=loss_cycle.item(),
                    loss_identity=loss_identity.item(), end='\r')
        if bx%100==0: generate_sample()
    log.report_avgs(epoch+1)
```

15. Generate the images once we have trained the models:

```
generate_sample()
```

The preceding code generates the following output:

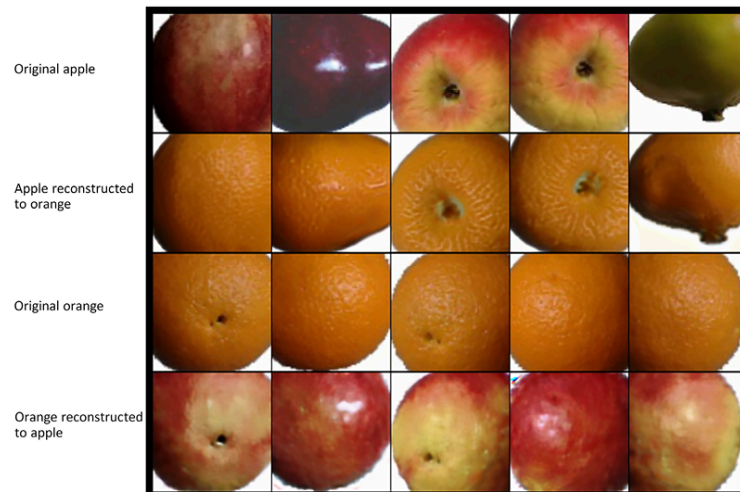


Figure 13.7: Original and reconstructed apples to oranges and vice versa

In the preceding image, we can see that we are successfully able to convert apples into oranges (the first two rows) and oranges into apples (the last two rows).

So far, we have learned about paired image-to-image translation through the Pix2Pix GAN and unpaired image-to-image translation through CycleGAN. In the next section, we will learn about leveraging StyleGAN to convert an image of one style into an image of another style.

Leveraging StyleGAN on custom images

In *Chapter 11*, we learned about neural style transfer. We generated an image by blending the style of one image with the content of another image. However, what if we want to create a younger version of a person in a picture or add certain attributes to an image, such as glasses? StyleGAN can do this. Let's learn how in the following sections.

The evolution of StyleGAN

Let's first look at a few developments prior to the invention of StyleGAN. As we know, generating fake faces (as we saw in the previous chapter) involves the usage of GANs. The biggest problem that research faced was that the images that could be generated were small (typically 64 x 64). Any effort to generate larger images caused the generators or discriminators to fall into local minima, which would stop training and generate gibberish. One of the major leaps in generating high-quality images appeared in a research paper that proposed **Progressive GAN (ProGAN)** – <https://arxiv.org/abs/1710.10196>, which used a clever trick. The size of both the generator and discriminator is progressively increased:

1. First, you create a generator and discriminator to generate 4 x 4 images from a latent vector.
2. Additional convolution (and upscaling) layers are then added to the trained generator and discriminator, which will be responsible for accepting the 4 x 4 images (which are generated from latent vectors in step 1) and generating/discriminating 8 x 8 images.
3. Next, new layers are created in the generator and discriminator once again so they can be trained to generate larger images. Step by step, the image size is increased in this way, the logic being that it is easier to add a new layer to an already well-functioning network than trying to learn all the layers from scratch.

In this manner, images are upscaled to a resolution of 1,024 x 1,024 pixels:

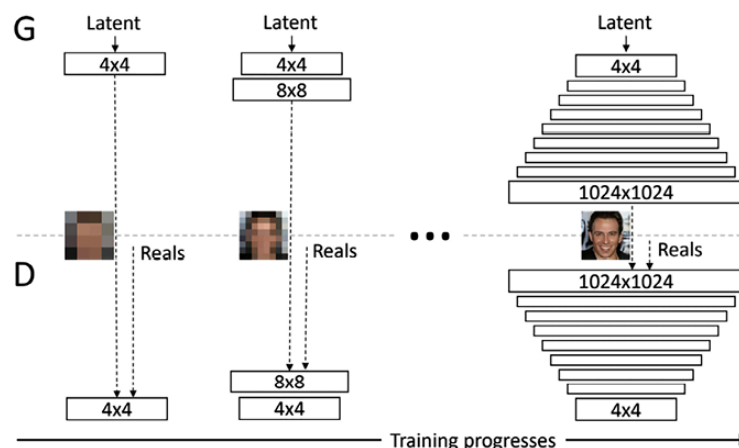


Figure 13.8: Image upscaling process

(source: <https://arxiv.org/pdf/1710.10196v3.pdf>)

As much as it succeeded, it was fairly difficult to control individual aspects of the generated image (such as gender and age), primarily because the network gets exactly one input (in the preceding image, this is **Latent** at the top of the network). StyleGAN addresses this problem. It uses a similar training scheme where images are progressively generated, but with an added set of latent inputs every time the network grows. This means the network now accepts multiple latent vectors at regular intervals (as shown in block (a) in Figure 13.9). Every latent given at a stage of generation dictates the features that are going to be generated at that stage of that network. Let's discuss the working details of StyleGAN in more detail here:

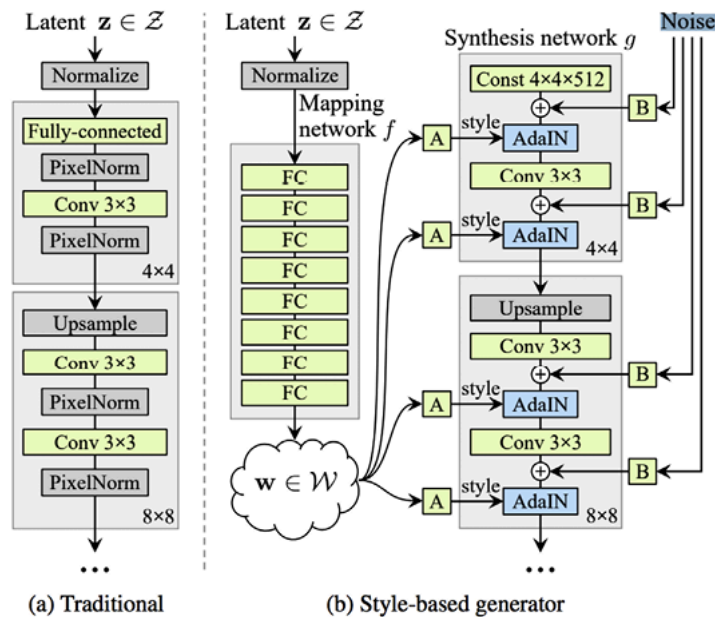


Figure 13.9: StyleGAN working details

(Source: <https://arxiv.org/pdf/1812.04948>)

In the preceding diagram, we can contrast the traditional way of generating images and the style-based generator. In a traditional generator, there is only one input. However, there is a mechanism in place within a style-based generator. Let's look at the details:

1. Create a random noise vector z of size 1×512 .
2. Feed this to an auxiliary network called the style network (or mapping network), which creates a tensor w of size 18×512 .
3. The generator (synthesis) network contains 18 convolution layers. Each layer will accept the following as inputs:
 1. The corresponding row of w (A)
 2. A random noise vector (B)
 3. The output from the previous layer

Note that noise (B) is given only for regularization purposes.

The preceding three combined will create a pipeline that takes in a 1×512 vector and creates a $1,024 \times 1,024$ image.

Let's now discuss how each of the $18 \times 1 \times 512$ vectors within the 18×512 vector that is generated from the mapping network contributes to the generation of an image:

The 1×512 vector that is added in the first few layers of the synthesis network contributes to the large-scale features present in the image such as pose and face shape (as they are responsible for generating the 4×4 , 8×8 images, and so on – which are the first few images that will be further enhanced in the later layers).

The vectors added in the middle layers correspond to small-scale features such as hairstyle and whether the eyes are open or closed (as they are responsible for generating the 16×16 , 32×32 , and 64×64 images).

The vectors added in the last few layers correspond to the color scheme and other microstructures of the image. By the time we reach the last few layers, the image structure is preserved, and the facial features are preserved but only image-level details such as lighting conditions are changed.

In the next section, we will leverage a pre-trained StyleGAN2 model to customize our image of interest to have different styles. For our objective, we will perform style transfer using the StyleGAN2 model. At a high level, here's how style transfer on faces works (the following will be clearer as you go through the results of the code):

- Say the `w1` style vector is used to generate `face-1` and the `w2` style vector is used to generate `face-2`. Both vectors have a shape of 18×512 .
- The first few of the 18 vectors in `w2` (which are responsible for generating images from 4×4 to 8×8 resolutions) are replaced with the corresponding vectors from `w1`. Then, we transfer very coarse features such as the pose from `face-1` to `face-2`.
- If the later style vectors (say the third to the fifteenth of the 18×512 vectors – which are responsible for generating 64×64 to 256×256 resolution images) are replaced in `w2` with those from `w1`, then we transfer features such as eyes, nose, and other mid-level facial features.
- If the last few style vectors (which are responsible for generating 512×512 to $1,024 \times 1,024$ resolution images) are replaced, fine-level features such as complexion and background (which don't affect the overall face in a significant manner) are transferred.

With an understanding of how style transfer is done, let's now see how to perform style transfer using StyleGAN2.

Implementing StyleGAN

To achieve style transfer on custom images using StyleGAN2, we follow these broad steps:

1. Take a custom image.
2. Align the custom image so that only the face region of the image is stored.
3. Fetch the latent vector that is likely to generate the custom aligned image when passed through StyleGAN2.
4. Generate an image by passing a random noise/latent vector (1 x 512) to the mapping network.

By this step, we have two images – our custom aligned image and the image generated by the StyleGAN2 network. We now want to transfer some of the features of the custom image to the generated image and vice versa. Let's code up the preceding strategy (note that we are leveraging a pre-trained network fetched from a GitHub repository, as training such a network takes days if not weeks):



You need a CUDA-enabled environment to run the following code. The following code is available as `Customizing_StyleGAN2.ipynb` in the `Chapter13` folder of this book's GitHub repository at <https://bit.ly/mcvcv-2e>. The code contains URLs to download data from and is moderately lengthy. We strongly recommend you execute the notebook in GitHub to reproduce results while you read the steps to perform and the explanations of the various code components in the text.

1. Clone the repository, install the requirements, and fetch the pre-trained weights:

```
import os
if not os.path.exists('pytorch_stylegan_encoder'):
    !git clone https://github.com/sizhky/pytorch_stylegan_encoder.git
    %cd pytorch_stylegan_encoder
    !git submodule update --init --recursive
    !wget -q https://github.com/jacobhallberg/pytorch_stylegan_encoder/releases/download/v1.0/
    !unzip -q trained_models.zip
    !rm trained_models.zip
    !pip install -qU torch_snippets
    !mv trained_models/stylegan_ffhq.pth InterFaceGAN/models/pretrain
else:
    %cd pytorch_stylegan_encoder

from torch_snippets import *
```

2. Load the pre-trained generator and the synthesis network, mapping the network's weights:

```
from InterFaceGAN.models.stylegan_generator import StyleGANGenerator
from models.latent_optimizer import PostSynthesisProcessing
synthesizer=StyleGANGenerator("stylegan_ffhq").model.synthesis
mapper = StyleGANGenerator("stylegan_ffhq").model.mapping
trunc = StyleGANGenerator("stylegan_ffhq").model.truncation
```

3. Define the function to generate an image from a random vector:


```

post_processing = PostSynthesisProcessing()
post_process = lambda image: post_processing(image)\
    .detach().cpu().numpy().astype(np.uint8)[0]
def latent2image(latent):
    img = post_process(synthesizer(latent))
    img = img.transpose(1,2,0)
    return img

```

4. Generate a random vector:

```

rand_latents = torch.randn(1,512).cuda()

```

In the preceding code, we are passing the random 1 x 512-dimensional vector through mapping and truncation networks to generate a vector that is 1 x 18 x 512. These 18 x 512 vectors are the ones that dictate the style of the generated image.

5. Generate an image from the random vector:

```

show(latent2image(trunc mapper(rand_latents))), sz=5)

```

The preceding code generates the following output:



Figure 13.10: Image from random latents

So far, we have generated an image. In the next few lines of code, you will learn about performing style transfer between the preceding generated image and an image of your choice.

6. Fetch a custom image (MyImage.jpg) and align it. Alignment is important to generate proper latent vectors because all generated images in StyleGAN have the face centered and features prominently visible:

```

!wget https://www.dropbox.com/s/lpw10qawsc5ipbn/MyImage.JPG\
-O MyImage.jpg
!git clone https://github.com/Puzer/stylegan-encoder.git
!mkdir -p stylegan-encoder/raw_images
!mkdir -p stylegan-encoder/aligned_images
!mv MyImage.jpg stylegan-encoder/raw_images

```

7. Align the custom image:

```
!python stylegan-encoder/align_images.py \
    stylegan-encoder/raw_images/ \
    stylegan-encoder/aligned_images/
!mv stylegan-encoder/aligned_images/* ./MyImage.jpg
```

8. Use the aligned image to generate latents that can reproduce the aligned image perfectly. This is the process of identifying the latent vector combination that minimizes the difference between the aligned image and the image generated from the latent vector:

```
from PIL import Image
img = Image.open('MyImage.jpg')
show(np.array(img), sz=4, title='original')
!python encode_image.py ./MyImage.jpg \
    pred_dlatents_myImage.npy \
    --use_latent_finder true \
    --image_to_latent_path ./trained_models/image_to_latent.pt
pred_dlatents = np.load('pred_dlatents_myImage.npy')
pred_dlatent = torch.from_numpy(pred_dlatents).float().cuda()
pred_image = latent2image(pred_dlatent)
show(pred_image, sz=4, title='synthesized')
```

The preceding code generates the following output:



Figure 13.11: Original image and the synthesized image from the corresponding latents

The `encode_image.py` Python script, at a high level, does the following (for a thorough understanding of each step, we encourage you to go through the script in the GitHub repo):

1. Creates a random vector in latent space. Alternatively, we can get a set of initial latents (vector) that require lesser number of optimizations by passing the original image through a network initialized with the weights `image_to_latent.pt` and architecture from the file `models/image_to_latent.py` in the same repo.

2. Synthesizes an image with this vector.
3. Compares the synthesized image with the original input image using VGG's perceptual loss.
4. Performs backpropagation on the w random vector to reduce this loss for a fixed number of iterations.
5. The optimized latent vector will now synthesize an image for which VGG gives near-identical features as the input image, and hence the synthesized image will look similar to the input image. We now have the latent vectors that correspond to the image of interest.
6. Perform style transfer between images. As discussed, the core logic behind style transfer is actually the transfer of parts of style tensors, that is, a subset of 18 of the 18 x 512 style tensors. Here, we will be transferring the first two rows (of the 18 x 512 tensors) in one case, 3-15 rows in one case, and 15-18 rows in one case. Since each set of vectors is responsible for generating different aspects of the image, each set of swapped vectors swaps different features in the image:

```
idxs_to_swap = slice(0,3)
my_latents=torch.Tensor(np.load('pred_dlatents_myImage.npy',
                                allow_pickle=True))

A, B = latent2image(my_latents.cuda()),
        latent2image(trunc(mapper(rand_latents)))
generated_image_latents = trunc(mapper(rand_latents))
x = my_latents.clone()
x[:,idxs_to_swap] = generated_image_latents[:,idxs_to_swap]
a = latent2image(x.float().cuda())
x = generated_image_latents.clone()
x[:,idxs_to_swap] = my_latents[:,idxs_to_swap]
b = latent2image(x.float().cuda())
subplots([A,a,B,b], figsize=(7,8), nc=2,
          subtitle='Transfer high level features')
```

The preceding code generates the following output:



Figure 13.12: Original images (left side) and the corresponding style transfer images (right side)

In Figure 13.12, since we are swapping very early in the pipeline, the most high-level features, such as age, are swapped. While swapping the next level of features (4 to 15), we will see that the next level of features, such as color palette and background, get swapped. Finally, the layers (15,18) don't seem to change the images at all since these features are very subtle and affect very fine details in the pictures, such as lighting. Here's the output with `idxs_to_swap` as `slice(4,15)` and `slice(15,18)` respectively.

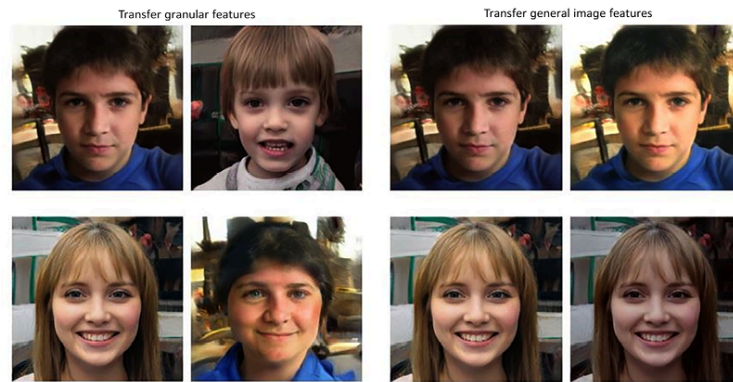


Figure 13.13: Style transfer with layer swaps at different levels

7. Next, we extrapolate a style vector so that the new vectors will only change the expression of our custom image. To do this, you need to compute the right direction to move the latent vector in. We can achieve this by first creating a lot of fake images. An SVM classifier is then used to identify whether the people within images are smiling or not. This SVM hence creates a hyperplane that separates smiling from non-smiling faces. The required direction to move is going to be normal to this hyperplane, which is presented as `stylegan_ffhq_smile_w_boundary.npy`. Implementation details can be found in the `InterfaceGAN/edit.py` code:

```
!python InterfaceGAN/edit.py\
-m stylegan_ffhq\
-o results_new_smile\
-b InterfaceGAN/boundaries/stylegan_ffhq_smile_w_boundary.npy\
-i pred_dlatents_myImage.npy\
-s WP\
--steps 20
generated_faces = glob.glob('results_new_smile/*.jpg')
subplots([read(im,1) for im in sorted(generated_faces)],
          figsize=(10,10))
```

Here's how the generated images look:



Figure 13.14: Progression of emotion from frown to smile

In summary, we have learned how research has progressed in generating very high-resolution images of faces using GANs. The trick is to increase the complexity of both the generator and discriminator in steps of increasing resolution so that at each step, both the models are decent at their tasks. We learned how you can manipulate the style of a generated image by ensuring that the features at every resolution are dictated by an independent input called a style vector. We also learned how to manipulate the styles of different images by swapping styles from one image to another.



VToonify can be used to generate high-quality artistic variations from an input video. The paper and associated code can be found here:

<https://github.com/williamyang1991/VToonify>.

Now that we have learned about leveraging the pre-trained StyleGAN2 model to perform style transfer, in the next section, we will leverage the pre-trained SRGAN model to generate images in high resolution.

Introducing SRGAN

In the previous section, we saw a scenario in which we used a pre-trained StyleGAN to generate images in a given style. In this section, we will take it a step further and learn about using pre-trained models to perform image super-resolution. We will gain an understanding of the architecture of the SRGAN model before implementing it on images.

First, we will explain why a GAN is a good solution for the task of super-resolution. Imagine a scenario in which you are given an image and asked to increase its resolution. Intuitively, you would consider various interpolation techniques to perform super-resolution. Here's a sample low-resolution image along with the outputs of various techniques:



Figure 13.15: The performance of different techniques of image super-resolution

(source: <https://arxiv.org/pdf/1609.04802.pdf>)

In the preceding image, we can see that traditional interpolation techniques such as bicubic interpolation do not help as much when reconstructing an image from a low resolution (in this case, a 4X down-scaled image of the original image).

While a super-resolution ResNet-based UNet could be useful in this scenario, GANs can be more useful as they simulate human perception. The discriminator, given that it knows what a typical super-resolution image looks like, can detect a scenario where the generated image has properties that do not necessarily look like an image with high resolution.

With the usefulness of GANs for super-resolution established, let's understand and leverage the pre-trained model.

Architecture

While it is possible to code and train an SRGAN from scratch, we will use pre-trained models where we can. Hence, for this section, we will leverage the model developed by Christian Ledig and his team and published in the paper titled *Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network*.

The architecture of an SRGAN is as follows:

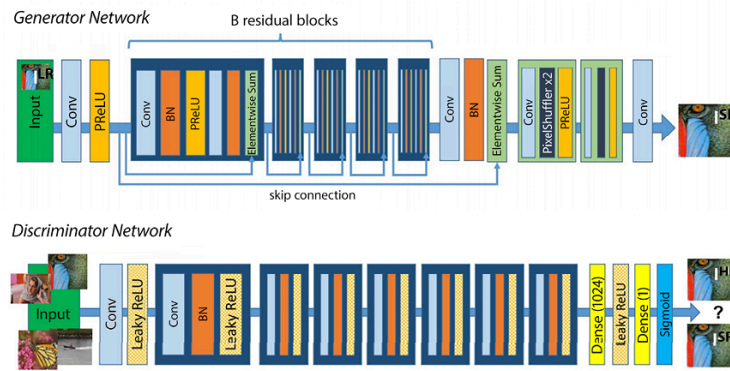


Figure 13.16: SRGAN architecture

(source: <https://arxiv.org/pdf/1609.04802.pdf>)

In the preceding image, we can see that the discriminator takes high-resolution images as input to train a model that predicts whether an image is a high-resolution or a low-resolution image. The generator network takes a low-resolution image as input and comes up with a high-resolution image. While training the model, both content loss and adversarial loss are minimized. For a detailed explanation of the details of model training and a comparison of the results obtained from the various techniques used to come up with high-resolution images, we recommend that you go through Ledig's paper.

With a high-level understanding of how the model is built, we will now code the way to leverage a pre-trained SRGAN model to convert a low-resolution image into a high-resolution image.

Coding SRGAN

Here are the steps for loading the pre-trained SRGAN and making our predictions:



The following code is available as `Image super resolution using SRGAN.ipynb` in the `Chapter13` folder of this book's GitHub repository: <https://bit.ly/mcvc-2e> The code contains URLs to download data from. We strongly recommend you execute the notebook in GitHub to reproduce results while you go through the steps to perform and the explanations of the various code components in the text.

1. Import the relevant packages and the pre-trained model:

```
import os
if not os.path.exists('srgan.pth.tar'):
    !pip install -q torch_snippets
    !wget -q https://raw.githubusercontent.com/sizhky/a-PyTorch-Tutorial-to-Super-Resolution/mi
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials
```

```

auth.authenticate_user()
gauth = GoogleAuth()
gauth.credentials = GoogleCredentials.get_application_default()
drive = GoogleDrive(gauth)
downloaded = drive.CreateFile({'id': \
    '1_PJ1Uimbr0xrPjE8U3Q_bG7XycGgsbVo'})
downloaded.GetContentFile('srgan.pth.tar')
from torch_snippets import *
device = 'cuda' if torch.cuda.is_available() else 'cpu'

```

2. Load the model:

```

model = torch.load('srgan.pth.tar', map_location='cpu')['generator'].to(device)
model.eval()

```

3. Fetch the image to convert to a higher resolution:

```

!wget https://www.dropbox.com/s/nmzwu68nr19j01f/Hema6.JPG

```

4. Define the functions to preprocess and postprocess the image:

```

preprocess = T.Compose([
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406],
                [0.229, 0.224, 0.225]),
    T.Lambda(lambda x: x.to(device))
])
postprocess = T.Compose([
    T.Lambda(lambda x: (x.cpu().detach()+1)/2),
    T.ToPILImage()
])

```

5. Load the image and preprocess it:

```

image = readPIL('Hema6.JPG')
image.size
# (260,181)
image = image.resize((130,90))
im = preprocess(image)

```

Note that, in the preceding code, we have performed an additional resize on the original image to further blur the image, but this is done only for illustration because the improvement is more visible when we use a down-scaled image.

6. Pass the preprocessed image through the loaded `model` and postprocess the output of the model:

```

sr = model(im[None])[0]
sr = postprocess(sr)

```

7. Plot the original and the high-resolution images:


```
subplots([image, sr], nc=2, figsize=(10,10),  
         titles=['Original image', 'High resolution image'])
```

The preceding code results in the following output:

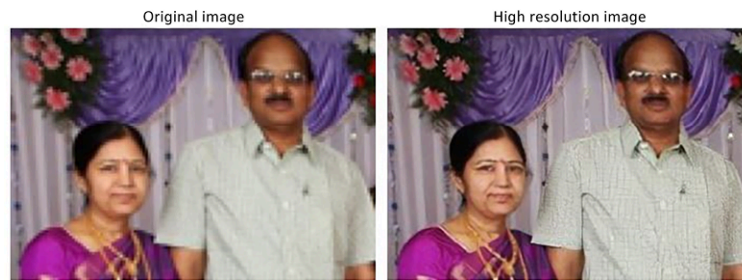


Figure 13.17: Original image and the corresponding SRGAN output

In the preceding image, we can see that the high-resolution image captured details that were blurred in the original image.

Note that the contrast between the original and the high-resolution image will be high if the original image is blurred or low resolution. However, if the original image is not blurred, the contrast will not be that high. We encourage you to work with images of varying resolutions.

Summary

In this chapter, we have learned about generating images from the contours of an image using the Pix2Pix GAN. Further, we learned about the various loss functions in CycleGAN to convert images of one class to another. Next, we learned about how StyleGAN can be used to generate realistic faces and also copy the style from one image to another, depending on how the generator is trained. Finally, we learned about using the pre-trained SRGAN model to generate high-resolution images. All of these techniques lay a strong foundation as we advance to learn about more modern ways of transferring image attributes in *Chapters 16* and *17*.

In the next chapter, we will switch gears and learn about combining computer vision techniques with other prominent techniques in reinforcement learning.

Questions

1. Why do we need a Pix2Pix GAN if a supervised learning algorithm such as UNet could have worked to generate images from contours?
2. Why do we need to optimize for three different loss functions in CycleGAN?
3. How do the tricks used by ProgressiveGAN help in building a StyleGAN model?
4. How do we identify the latent vectors that correspond to a given custom image?

Learn more on Discord

Join our community's Discord space for discussions with the authors and other readers:

<https://packt.link/modcv>

